RESPIRATORY DISEASE
MULTIMODAL CLASSIFICATION PIPELINE

Technical Documentation & Code Reference

Version	1.0.0 — Production Release
Framework	TensorFlow / Keras
Target Classes	Asthma, COPD, Healthy, Pneumonia
Modalities	Image + Audio + Tabular Clinical Data

1. Executive Summary

This document provides complete technical documentation for the Respiratory Disease Multimodal Classification Pipeline — an end-to-end deep learning system designed to classify patients into four respiratory health categories: Asthma, COPD, Healthy, and Pneumonia.

The system fuses three independent data modalities into a single unified prediction:

- Chest X-ray images (NIH ChestX-ray14 dataset) processed through a DenseNet-121 CNN backbone
- Respiratory audio recordings (ICBHI dataset) converted to mel spectrograms and processed through EfficientNet-B0
- Clinical tabular data (13 structured patient features) processed through a custom Residual MLP

Each modality is trained independently as a specialist branch. The learned feature vectors from all three branches are concatenated and passed through a joint fusion network that produces the final diagnosis. The system generates a structured clinical report for each patient, including probability scores for all four classes and a set of evidence-based clinical recommendations.

> *Key Technical Highlights: This pipeline uses late feature fusion (not late decision fusion), meaning the branch models share feature representations — not just their final probability outputs — which allows the fusion network to learn cross-modal interactions.*

Key Performance Targets

Architecture	Three-branch late fusion multimodal neural network
Image Branch	DenseNet-121, top-20 layers unfrozen, ImageNet pretrained
Audio Branch	EfficientNet-B0, frozen backbone, ImageNet pretrained
Tabular Branch	Custom residual MLP with GaussianNoise, BatchNorm, GELU
Fusion Strategy	Concatenation of pre-softmax feature vectors → Dense(256) → Dense(128) → Softmax(4)
Training	50 max epochs, EarlyStopping, AdamW optimizer, class-weighted loss
Datasets	NIH ChestX-ray14 (image), ICBHI (audio), custom clinical CSV (tabular)
Classes	asthma (0), copd (1), healthy (2), pneumonia (3)
Output	Structured doctor report with per-class probabilities and clinical flags

2. Project Overview

2.1 Problem Statement

Respiratory diseases — particularly COPD, asthma, and pneumonia — are among the leading causes of morbidity and mortality worldwide. Early and accurate diagnosis requires integrating multiple clinical signals: a physician examines X-rays, listens to lung sounds, and reviews patient history and biomarkers. Traditional single-modality automated systems replicate only one of these channels, leaving significant diagnostic information on the table.

This pipeline addresses this by building a system that simultaneously processes all three clinical modalities and fuses them into a single prediction — closer to how a real physician reasons.

2.2 Objectives

1. Train specialist deep learning models on each modality independently.
2. Align samples across all three datasets to form matched patient triplets (image + audio + tabular row).
3. Build a joint fusion model that takes features from all three branches and produces a unified 4-class prediction.
4. Generate per-patient clinical reports with confidence scores, uncertainty flags, and clinical recommendations.
5. Save all models in multiple formats (.keras, .h5, .joblib) for deployment flexibility.

2.3 Target Classes

Class	Label Index	Description
`asthma`	0	Mapped from NIH 'Atelectasis'; chronic inflammatory airway disease
`copd`	1	Mapped from NIH 'Emphysema'; chronic obstructive pulmonary disease
`healthy`	2	Mapped from NIH 'No Finding'; no acute respiratory pathology
`pneumonia`	3	Mapped directly from NIH 'Pneumonia'

2.4 Datasets Used

NIH ChestX-ray14	Chest X-ray images (224×224 PNG). Labels mapped from 14 pathology categories down to 4 master classes. Healthy class capped at 2,000 samples to prevent imbalance.

ICBHI 2017	Respiratory lung sound recordings (.wav). Metadata from patient_diagnosis.csv maps patient IDs to diagnoses. Audio converted to mel spectrogram images.
clinical_data.csv	User-supplied CSV with 13 structured clinical features per patient (age, BMI, FEV1, SpO2, etc.). Excludes bronchitis and lung_cancer classes from training.

2.5 Global Configuration

MASTER_CLASSES	['asthma', 'copd', 'healthy', 'pneumonia']
NUM_CLASSES	4
IMG_SIZE	(224, 224) — matches ImageNet pretrained input requirements
BATCH_SIZE	16
EPOCHS	50 (with early stopping)
SEED	42 — applied to random, numpy, and tensorflow for reproducibility
TABULAR_FEATURES	13 clinical features used as model input (age, gender_enc, smoking_status, fev1_percent, spo2, respiratory_rate, cough_severity, wheeze, chest_tightness, crackles, fever, bmi, copd_exacerbations)
CLASSES_TO_DROP	['bronchitis', 'lung_cancer'] — excluded from tabular training
FUSION_WEIGHTS	{'image': 0.50, 'tabular': 0.35, 'audio': 0.15} — defined but not used in late fusion architecture (for reference only)

3. System Architecture

3.1 High-Level Pipeline

The pipeline is structured in six sequential stages:

6. Data Ingestion — Download and prepare NIH image data, ICBHI audio data, and clinical tabular data.
7. Branch Training — Train each of the three specialist models independently on their respective modalities.
8. Multimodal Alignment — Build matched triplets (one image + one audio file + one tabular row per class) to create a unified dataset for fusion training.
9. Fusion Model Training — Connect the three trained branches, extract pre-softmax feature vectors, concatenate, and train a joint reasoning head.
10. Evaluation & Reporting — Evaluate on the held-out test set, generate confusion matrices, ROC-AUC curves, and per-patient doctor reports.
11. Model Serialization — Save all models in .keras, .h5, and .joblib formats; zip and export artifacts.

3.2 Model Architecture Detail

Image Branch — DenseNet-121

DenseNet-121 is a convolutional neural network where each layer receives feature maps from all preceding layers via dense connections. This architecture is particularly well-suited for chest X-ray analysis because:

- Dense connections provide strong gradient flow, reducing vanishing gradient problems.
- Feature reuse makes the network highly parameter efficient.
- The architecture performs well even with relatively limited medical imaging datasets.

In this pipeline, the top 20 layers of DenseNet-121 are unfrozen for fine-tuning, while the lower layers remain frozen to preserve general ImageNet features. The output is passed through GlobalAveragePooling2D, a Dropout(0.4) layer (the feature vector extracted for fusion), and a final Dense(4, softmax) head for standalone training.

Audio Branch — EfficientNet-B0

EfficientNet-B0 is a computationally efficient CNN that scales depth, width, and resolution uniformly. Its backbone is completely frozen in this pipeline. Audio files are converted to mel spectrogram images before being fed to the model, allowing an image classification backbone to process acoustic data. The mel spectrogram conversion captures the frequency content of respiratory sounds over time — a standard approach for respiratory sound classification.

Tabular Branch — Residual MLP

Clinical tabular features are processed through a custom deep neural network with residual (skip) connections — inspired by ResNet — applied to fully connected layers. This prevents degradation when training deeper networks on structured data. Key design choices:

- GaussianNoise(0.02) applied to inputs as a regularizer (acts as data augmentation for tabular data).
- MaxNorm weight constraints prevent weight explosion without explicit gradient clipping.
- GELU activation is used throughout (smoother than ReLU, better suited for transformer-style networks).
- MixUp augmentation generates 1,200 synthetic training samples by interpolating between real samples.

Fusion Architecture — Late Feature Fusion

The fusion model connects to all three branch models simultaneously. Crucially, it extracts the feature vectors from layer[-2] of each branch (the Dropout layer immediately before the final softmax), NOT the probability outputs. This means the fusion network sees raw learned features — not rounded probabilities — giving it far more information to work with.

The concatenated feature vector [img_features | aud_features | tab_features] is passed through:

- Dense(256) → BatchNorm → GELU → Dropout(0.4)
- Dense(128) → BatchNorm → GELU → Dropout(0.3)
- Dense(4, softmax) — the final unified prediction

> ⚠ *All three branch models are frozen (trainable=False) during fusion training. Only the new Dense(256) → Dense(128) → Dense(4) head layers are trained. This prevents catastrophic forgetting in the branch models.*

3.3 Directory Structure

```
respiratory_global_pipeline/
  data/
    nih_chestxray/     ← NIH X-ray images and metadata CSV
    icbhi/             ← ICBHI .wav files and patient_diagnosis.csv
  models/              ← Saved .keras, .h5, .joblib model files
  reports/             ← Confusion matrices, training history plots, ROC curves
```

4. Features and Functionality

4.1 Data Ingestion & Preparation

Automated Kaggle Dataset Download

The pipeline automatically downloads datasets from Kaggle using the kaggle CLI. The kaggle.json API key file is expected to be present in the working directory and is moved to ~/.kaggle/ for authentication. Both datasets are downloaded and extracted to their respective subdirectories under data/. The download is guarded by a directory-empty check — if files already exist, the download is skipped.

NIH Chest X-ray Label Mapping

The NIH dataset contains 14 disease labels. Four of these are mapped to the pipeline's master classes:

NIH Original Label	Mapped Class	Clinical Rationale
Emphysema	copd	Emphysema is a primary form of COPD
Pneumonia	pneumonia	Direct mapping
Atelectasis	asthma	Partial lung collapse overlaps clinically with asthma
No Finding	healthy	No pathological finding present

> ⚠ *The Atelectasis → asthma mapping is a simplification for this educational pipeline. In clinical practice, atelectasis and asthma are distinct conditions. This mapping should be reconsidered for production medical deployment.*

Class Balancing

The NIH dataset contains substantially more 'No Finding' (healthy) samples than disease samples. The pipeline caps healthy samples at 2,000 using random sampling, preventing the model from becoming biased towards predicting healthy.

Exploratory Data Analysis

The pipeline uses ydata-profiling (formerly pandas-profiling) to auto-generate a comprehensive HTML profile report of the tabular clinical data. This report includes: missing value analysis, feature distributions, correlation heatmaps, data type information, and interaction plots. The report is saved to reports/tabular_data_profile_report.html.

4.2 Image Processing

CLAHE Preprocessing

Each chest X-ray is preprocessed using CLAHE (Contrast Limited Adaptive Histogram Equalization) before being fed to the model. CLAHE is a local contrast enhancement algorithm that:

- Divides the image into small tiles (8×8 grid in this pipeline).
- Applies histogram equalization to each tile independently.
- Clips the histogram at a limit (clipLimit=2.0) to prevent over-amplification of noise.

This significantly improves the visibility of subtle features in chest X-rays (such as consolidation, ground-glass opacities) which often have low natural contrast.

Data Augmentation (Training Only)

During training, the NIHImageGenerator applies online augmentation to each image individually using ImageDataGenerator:

- rotation_range=10 — slight rotation to handle patient positioning variation.
- width_shift_range=0.08 — horizontal translation for position invariance.
- height_shift_range=0.08 — vertical translation.
- zoom_range=0.08 — scale invariance.
- horizontal_flip=True — chest X-rays can be read flipped (mirrored).

Augmentation is only applied to training batches, never to validation or test batches.

4.3 Audio Processing

Mel Spectrogram Conversion

Audio files (.wav) are converted to 2D mel spectrogram images before being passed to EfficientNet-B0. The conversion pipeline:

12. Load audio with librosa at 22,050 Hz, mono channel, clipped/padded to 5 seconds.
13. Compute mel spectrogram with 128 mel bands, max frequency 2,000 Hz (captures respiratory sound range).
14. Convert power spectrogram to decibel scale using librosa.power_to_db().
15. Normalize the dB values to [0, 255] range, cast to uint8.
16. Resize to (224, 224) to match the EfficientNet input.
17. Stack single channel × 3 to create a 3-channel RGB-like tensor.
18. Normalize back to [0, 1] float32.

4.4 Tabular Feature Engineering

Feature Set

13 clinical features are used:

age	Patient age in years
gender_enc	Binary encoding: 1 = Male, 0 = Female
smoking_status	Smoking status (encoded)
fev1_percent	FEV1 % predicted — key indicator of obstructive lung disease

spo2	Blood oxygen saturation (normal > 95%)
respiratory_rate	Breaths per minute
cough_severity	Cough severity score
wheeze	Presence of wheezing (binary/ordinal)
chest_tightness	Chest tightness score
crackles	Presence of crackles in lung auscultation
fever	Presence/degree of fever
bmi	Body Mass Index
copd_exacerbations	Number of past COPD exacerbations

MixUp Augmentation

The pipeline generates 1,200 synthetic training samples using the MixUp technique. For each synthetic sample, two real samples A and B are randomly selected, and a mixing coefficient λ is drawn from a Beta(0.3, 0.3) distribution. The synthetic sample is: $X_mix = \lambda \cdot X_A + (1-\lambda) \cdot X_B$. The label is determined by taking argmax of the mixed one-hot vectors, effectively assigning the class that contributes more to the mixture. This helps regularize the decision boundary and improves generalization.

4.5 Multimodal Alignment — Triplet Construction

A key challenge in multimodal learning is that the three datasets contain different patients with no natural correspondence. The pipeline solves this by constructing artificial triplets:

19. For each class, collect all image paths, audio paths, and tabular rows belonging to that class.
20. Determine min_count = min(num_images, num_audios, num_rows) — the bottleneck modality.
21. Randomly shuffle each list independently.
22. Pair the i-th image, i-th audio file, and i-th tabular row together for i in range(min_count).

This means each training sample is (random_image_of_class_X, random_audio_of_class_X, random_tabular_row_of_class_X). The three items don't belong to the same patient — they're matched only by class label. This is an accepted approach in multimodal research when cross-modal patient correspondence doesn't exist.

> ⚠ *The audio branch (ICBHI) typically has the fewest samples per class, making it the bottleneck modality that determines the total triplet count. This significantly limits the fusion training set size.*

4.6 Model Evaluation

Evaluation Metrics

All branches and the global model are evaluated using:

- Per-class Precision, Recall, F1-score via classification_report.
- Macro-averaged F1-score via f1_score(average='macro') — treats all classes equally regardless of support.

- Confusion matrix (visualized as seaborn heatmap and saved as PNG).
- ROC-AUC curve for each class (one-vs-rest binarization).
- Precision-Recall curve with Average Precision score for each class.

Clinical Uncertainty Flagging

The doctor report includes a high_uncertainty flag that is set to True when the model's confidence (max softmax probability) is below 65%. When this flag is raised, human_review_needed is also set to True. This is a safety mechanism to ensure borderline cases are escalated to a clinician rather than acted on automatically.

4.7 Model Persistence

Models are saved in three formats for maximum compatibility:

.keras format	Native TensorFlow 2.x format. Preferred for re-loading and further training within TensorFlow.
.h5 format	Legacy HDF5 format. Compatible with older TensorFlow/Keras versions and some deployment platforms.
.joblib format	Serialized using joblib. Note: this works for sklearn objects (like the StandardScaler). For Keras models, joblib dump may produce inconsistent results vs. model.save() — the .keras/.h5 formats are preferred for neural networks.
tabular_scaler	The StandardScaler fitted on training tabular data is saved as tabular_scaler.joblib. This MUST be used at inference time to normalize input features identically to training.

5. Line-by-Line Function Explanation

This section provides a detailed walkthrough of every function and class in the pipeline.

5.1 load_and_filter_nih_metadata()

Purpose	Load NIH chest X-ray metadata, apply label mapping, cap healthy samples, attach file paths, and return a filtered DataFrame.

```
meta_path = next(image_dir.rglob('Data_Entry_2017*.csv'), None)
```

Recursively search the image directory for a CSV file matching the NIH metadata filename pattern. rglob('*.csv') traverses all subdirectories. next(..., None) returns None instead of raising StopIteration if no file is found — allowing a graceful empty-DataFrame return.

```
df = df[df['Finding Labels'].isin(NIH_LABEL_MAP.keys())].copy()
```

Filter the DataFrame to keep only rows where the 'Finding Labels' column contains one of the four mapped labels (Emphysema, Pneumonia, Atelectasis, No Finding). .copy() prevents SettingWithCopyWarning by creating an independent copy of the filtered slice.

```
df['label'] = df['Finding Labels'].map(NIH_LABEL_MAP).str.lower()
```

Apply the NIH_LABEL_MAP dictionary to translate each NIH label to the master class string. .str.lower() normalizes to lowercase (redundant here since values are already lowercase, but a safe guard).

```
path_lookup = {p.name: str(p) for p in image_dir.rglob('*.png')}
```

Build a dictionary mapping each PNG filename (e.g., '00000001_000.png') to its full absolute path. This is a performance optimization — building the lookup once and using dictionary access is O(1) per lookup vs. O(n) for searching repeatedly.

```
df['filepath'] = df['Image Index'].map(path_lookup)
```

Map the 'Image Index' column (which contains filenames like '00000001_000.png') to full paths using the dictionary. Files not found in the lookup map to NaN.

```
df = df.dropna(subset=['filepath'])
```

Remove rows where the image file was not found on disk (NaN filepath). This handles cases where the metadata references files that weren't downloaded.

```
if len(healthy_df) > healthy_limit:     healthy_df =
healthy_df.sample(n=healthy_limit, random_state=SEED)
```

If the number of 'No Finding' (healthy) images exceeds the healthy_limit parameter (default: 2,000), randomly down-sample to exactly that limit. random_state=SEED ensures reproducibility.

```
df = pd.concat([healthy_df, diseased_df]).sample(frac=1,
random_state=SEED).reset_index(drop=True)
```

Concatenate the balanced healthy samples with all disease samples, then shuffle the entire DataFrame (sample(frac=1) selects 100% of rows in random order). reset_index(drop=True) resets the row indices to 0, 1, 2... after shuffling.

```
df['label_enc'] = MASTER_LE.transform(df['label'])
```

Apply the pre-fitted LabelEncoder to convert string class labels to integer indices (asthma=0, copd=1, healthy=2, pneumonia=3). MASTER_LE must be fitted before this call, which is done at the top of the notebook.

5.2 preprocess_chest_xray()

Purpose	Load a chest X-ray, apply CLAHE contrast enhancement, convert to 3-channel RGB, resize to target dimensions, and normalize to [0, 1].

```
img_gray = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
```

Load the image in grayscale mode (single channel). Chest X-rays are naturally grayscale — loading as color would create redundant identical channels without this explicit CLAHE step.

```
if img_gray is None: raise ValueError(f'Cannot load: {image_path}')
```

cv2.imread returns None (not an exception) on failure. This explicit check raises a meaningful ValueError instead of causing a confusing NoneType AttributeError later in the pipeline.

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)) img_clahe =
clahe.apply(img_gray)
```

Create a CLAHE object with a clip limit of 2.0 and a tile grid of 8×8 tiles. The clip limit controls how aggressively local contrast is enhanced — higher values increase contrast but also amplify noise. 2.0 is the standard clinical imaging setting. apply() performs the actual enhancement on the grayscale image.

```
img_rgb = cv2.cvtColor(img_clahe, cv2.COLOR_GRAY2RGB)
```

Convert the single-channel grayscale image to a 3-channel RGB image by duplicating the channel three times. This is required because DenseNet-121 expects 3-channel input (it was pretrained on RGB ImageNet images). The three channels are identical but necessary for shape compatibility.

```
img_res = cv2.resize(img_rgb, target_size, interpolation=cv2.INTER_AREA)
```

Resize the image to (224, 224). INTER_AREA is the preferred interpolation mode for downsampling — it calculates the average of pixels in the source region, preserving overall intensity and reducing aliasing artifacts compared to bilinear or nearest-neighbor interpolation.

```
return (img_res / 255.0).astype(np.float32)
```

Normalize pixel values from [0, 255] integer range to [0.0, 1.0] float32. This is required for neural network training stability and matches the input normalization expected by ImageNet pretrained models.

5.3 NIHImageGenerator (class)

Purpose	A tf.keras.utils.Sequence subclass that lazily loads, preprocesses, and optionally augments chest X-ray images in batches during training.

```
class NIHImageGenerator(tf.keras.utils.Sequence):
```

Subclassing tf.keras.utils.Sequence guarantees thread-safe batch generation and enables multiprocessing during training. Keras requires __len__ and __getitem__ to be defined.

```
self.indices = np.arange(len(self.df)) if self.shuffle:
np.random.shuffle(self.indices)
```

Create an index array [0, 1, 2, ..., n-1] and shuffle it at initialization if shuffle=True. This is later used in __getitem__ to map batch positions to actual row positions, allowing shuffled access without modifying the underlying DataFrame.

```
self.aug = ImageDataGenerator(rotation_range=10, ...)
```

Create an ImageDataGenerator object configured for mild augmentation. This is only used when augment=True (training). The instance is created once in __init__ for efficiency rather than re-creating it every batch.

```
def __len__(self):     return int(np.ceil(len(self.df) / self.batch_size))
```

Return the total number of batches. np.ceil ensures the last incomplete batch is included (e.g., 100 samples / 16 = 6.25 → 7 batches). This means the final batch may have fewer than batch_size samples.

```
batch_idx = self.indices[idx * self.batch_size: (idx + 1) * self.batch_size]
```

Slice the shuffled indices array to get the row indices for this batch. Using the indirection through self.indices (rather than directly slicing the DataFrame) is what makes shuffle-without-copy work.

```
X = np.zeros((len(batch_df), *IMG_SIZE, 3), dtype=np.float32)
```

Pre-allocate the full output array before the loop. This is a performance optimization — appending to a list and calling np.array() at the end creates unnecessary copies. np.zeros allocates exactly once.

```
try:     img = preprocess_chest_xray(row['filepath']) except:     img =
np.zeros((*IMG_SIZE, 3), dtype=np.float32)
```

Silently handle corrupted or missing image files by substituting a black (all-zero) image. This prevents training crashes due to data quality issues, at the cost of introducing a meaningless sample. In production, corrupted files should be logged and removed from the dataset.

```
if self.augment:     img = self.aug.random_transform(img)
```

Apply a random augmentation transform to the image if augment=True. random_transform() applies a combination of the configured augmentations stochastically — each call produces a different result. This is applied per-image rather than per-batch for maximum randomness.

```
y = to_categorical(y, num_classes=NUM_CLASSES)
```

Convert integer class labels [0, 1, 2, 3] to one-hot encoded vectors [[1,0,0,0], [0,1,0,0], ...]. This is done once at the end of the batch (rather than per-sample) as a single array operation for efficiency. Required for categorical_crossentropy loss.

```
def on_epoch_end(self):     if self.shuffle:
np.random.shuffle(self.indices)
```

Called automatically by Keras at the end of each epoch. Re-shuffling the indices array ensures that each epoch presents batches in a different order, which is critical for training generalization.

5.4 build_image_model()

Purpose	Construct and compile the DenseNet-121 image classification model with partial fine-tuning.

```
base = DenseNet121(include_top=False, weights='imagenet',
input_shape=(*IMG_SIZE, 3))
```

Load DenseNet-121 without its ImageNet classification head (include_top=False). weights='imagenet' downloads pretrained weights. input_shape=(*IMG_SIZE, 3) = (224, 224, 3) sets the expected input dimensions.

```
base.trainable = False for layer in base.layers[-20:]:     layer.trainable =
True
```

First freeze the entire backbone, then selectively unfreeze the last 20 layers. The early layers have learned general features (edges, textures) that transfer well from ImageNet. The last 20 layers learn higher-level, domain-specific features — unfreezing these allows fine-tuning to chest X-ray characteristics while preserving the stable lower-level representations.

```
x = base(inputs, training=False)
```

Pass inputs through the base model. training=False is important — it keeps batch normalization layers in inference mode (using running statistics) even during training. Without this, the partially frozen BN layers would update their statistics from the small batch, which can be harmful.

```
x = GlobalAveragePooling2D()(x)
```

Reduce the 3D feature maps (H, W, C) from the last DenseNet block to a 1D feature vector by averaging across the spatial dimensions. This produces a fixed-size vector regardless of input image size, and serves as a spatially compact representation.

```
x = Dropout(0.4)(x)
```

Apply 40% dropout to the feature vector. This is the layer whose output is extracted as the feature vector for fusion (layers[-2]). Dropout here acts as regularization preventing the model from memorizing training patterns.

```
optimizer=AdamW(learning_rate=1e-4, weight_decay=1e-4)
```

AdamW is Adam with decoupled weight decay regularization. Standard Adam applies weight decay coupled with the gradient, which can interfere with adaptive learning rates. AdamW decouples these, providing more effective regularization. weight_decay=1e-4 is a mild regularization penalty.

5.5 train_image_model()

Purpose	End-to-end training loop for the image branch: split data, compute class weights, train with callbacks, evaluate, and save.

```
train_df, test_df = train_test_split(df, test_size=0.20,
stratify=df['label_enc'])
```

Split data 80/20 for training and test. stratify=df['label_enc'] ensures both splits have the same class distribution as the original dataset — critical when class counts are unequal to prevent the test set from being all one class.

```
train_df, val_df = train_test_split(train_df, test_size=0.15,
stratify=train_df['label_enc'])
```

Further split training data: 15% of the 80% training set becomes validation. This gives roughly 68% / 12% / 20% for train/val/test respectively.

```
cw = compute_class_weight('balanced', classes=np.unique(y_train), y=y_train)
cw_dict = {int(c): w for c, w in zip(np.unique(y_train), cw)}
```

Compute class weights inversely proportional to class frequency. For a class with N_i samples out of N total: weight_i = N / (K × N_i), where K is the number of classes. These weights are passed to model.fit() to penalize the loss more strongly for misclassifications of minority classes.

```
EarlyStopping(monitor='val_loss', patience=4, restore_best_weights=True)
```

Stop training if the validation loss does not improve for 4 consecutive epochs. restore_best_weights=True means the model weights at the best-performing epoch (not the final epoch) are retained after training stops. This is crucial — without it, the model may overfit during the patience window.

```
ModelCheckpoint(str(MODEL_DIR / 'best_image_model.keras'), monitor='val_loss',
save_best_only=True)
```

Save the model to disk whenever validation loss improves. save_best_only=True means only the all-time best model is kept on disk (not every improvement). This is useful if training crashes before EarlyStopping fires.

5.6 wav_to_melspectrogram()

Purpose	Convert a .wav audio file into a 3-channel mel spectrogram image tensor of shape (224, 224, 3).

```
y, _ = librosa.load(filepath, sr=sr, duration=duration, mono=True)
```

Load the audio file with librosa. sr=22050 resamples all files to 22,050 Hz for consistency. duration=5.0 truncates or pads to exactly 5 seconds. mono=True mixes down multi-channel audio to single channel. The underscore discards the sample rate return value.

```
target_len = int(sr * duration) y = np.pad(y, (0, max(0, target_len - len(y))),
mode='constant')[:target_len]
```

Compute the required sample count (22050 × 5 = 110,250 samples). np.pad appends zeros to the end if the audio is shorter than 5 seconds (zero-padding = silence). [:target_len] truncates if longer than 5 seconds. Both operations ensure exactly target_len samples.

```
mel = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=n_mels, fmax=fmax)
```

Compute the mel spectrogram: compute the Short-Time Fourier Transform (STFT), square the magnitudes to get power, then map frequency bins to the mel scale (which approximates human auditory perception). n_mels=128 mel frequency bands, fmax=2000 Hz (captures the relevant 100–2000 Hz range for respiratory sounds).

```
mel_db = librosa.power_to_db(mel, ref=np.max)
```

Convert power values to decibel (log) scale: dB = 10 × log10(S / ref). ref=np.max normalizes so the maximum power becomes 0 dB. Log scale makes the spectrogram visually more informative and numerically more stable for neural networks.

```
mel_n = ((mel_db - mel_db.min()) / (mel_db.max() - mel_db.min() + 1e-8) *
255).astype(np.uint8)
```

Min-max normalize the dB values to [0, 255] for image representation. The + 1e-8 epsilon prevents division by zero for silent or near-silent audio. Cast to uint8 for cv2.resize compatibility.

```
return np.stack([mel_r] * 3, axis=-1).astype(np.float32) / 255.0
```

Stack the single-channel spectrogram image three times along the last axis to create a (224, 224, 3) tensor matching EfficientNet's expected input. Divide by 255 to normalize back to [0, 1] float32.

5.7 build_audio_model()

Purpose	Build the EfficientNet-B0 audio classification model with a fully frozen backbone.

```
base = EfficientNetB0(include_top=False, weights='imagenet',
input_shape=(*IMG_SIZE, 3)) base.trainable = False
```

Load EfficientNet-B0 with all layers frozen. Unlike the image model, the audio model's backbone is fully frozen because EfficientNet is more sensitive to fine-tuning instability on small medical audio datasets. The pre-trained features are reused as general texture/pattern detectors applied to spectrogram images.

```
optimizer=AdamW(learning_rate=5e-4, weight_decay=1e-4)
```

A higher learning rate (5e-4 vs. 1e-4 for the image model) is used because only the new classification head layers (not the frozen backbone) are being trained. Fewer trainable parameters generally allow a higher learning rate without divergence.

5.8 load_icbhi_metadata()

Purpose	Load ICBHI audio file paths and match them to diagnoses from patient_diagnosis.csv.

```
diag_df = pd.read_csv(diag_path, header=None, names=['patient_id',
'diagnosis'])
```

Read the diagnosis CSV with no header row (header=None), manually naming the two columns. The ICBHI patient_diagnosis.csv has format: patient_id, diagnosis — with no column headers.

```
pid = int(wav.stem.split('_')[0])
```

Extract the patient ID from the .wav filename. ICBHI filenames follow the format: {patient_id}_{recording_index}_{...}.wav. Splitting on '_' and taking index [0] gives the patient ID, which is cast to int for matching against the integer patient_id column in diag_df.

```
df = pd.DataFrame(records).drop_duplicates(subset='filepath')
```

Remove any duplicate file paths that may have appeared due to multiple recordings per patient being matched to the same diagnosis. This ensures each audio file appears exactly once in the dataset.

5.9 mixup_tabular()

Purpose	Generate synthetic tabular training samples using the MixUp interpolation technique.

```
lambdas = rng.beta(alpha, alpha, size=n_synthetic)
```

Sample n_synthetic mixing coefficients from a Beta(α, α) distribution. With α=0.3, this distribution is U-shaped — most samples are drawn near 0 or 1, meaning most synthetic samples are heavily dominated by one of the two parent samples. This creates subtle variations near the training data rather than many purely interpolated midpoints.

```
X_mix = lam * X[idx_a] + (1 - lam) * X[idx_b]
```

Create synthetic features by linearly interpolating between two randomly selected training samples. lam is shaped (n_synthetic, 1) via [:, None] to broadcast correctly against the 2D feature matrix.

```
oh_a = to_categorical(y[idx_a], num_classes=NUM_CLASSES) oh_b =
to_categorical(y[idx_b], num_classes=NUM_CLASSES) y_mix = np.argmax(lambdas[:,
None] * oh_a + (1 - lambdas[:, None]) * oh_b, axis=1)
```

Compute mixed one-hot labels by interpolating the one-hot representations of both parent classes, then taking argmax to assign a hard label. This assigns the label of the parent sample that contributed the larger mixing proportion. Note: in original MixUp, soft labels are used directly during training — using argmax here converts back to hard labels, which is a simplification but works well for the tabular DNN with sparse_categorical_crossentropy loss.

5.10 residual_mlp_block()

Purpose	A residual block for the tabular MLP that adds a skip connection around two Dense layers to prevent gradient degradation.

```
skip = x ... x = Add()([x, skip])
```

Save the input tensor as 'skip' before the transformation layers. After the two Dense + BatchNorm transformations, add the skip connection back to the transformed output. The intuition: the block learns the residual (difference) from the skip, rather than the full transformation — if no transformation is needed, the block can learn to produce near-zero residuals and the skip passes through intact.

```
kernel_constraint=MaxNorm(max_value=3.0)
```

Constrain the L2 norm of each weight vector to be at most 3.0 after every update. This prevents any single weight from growing unboundedly large, which can destabilize training. MaxNorm is particularly useful with tabular data where feature scales can vary widely.

```
use_bias=False
```

Disable the bias term in Dense layers that are followed by BatchNormalization. BatchNorm already applies a learnable shift parameter (beta) that serves the same purpose as a bias — including both would be redundant and waste parameters.

5.11 build_tabular_dnn()

Purpose	Build the complete Residual MLP for tabular clinical data processing.

```
x = GaussianNoise(0.02)(inputs)
```

Add Gaussian noise with standard deviation 0.02 to the inputs during training (disabled automatically during inference). This acts as a regularizer — by slightly perturbing the input features, the model is forced to be robust to small measurement errors and noise, which is

realistic for clinical data. The 0.02 std is very mild — preserving the clinical signal while adding robustness.

```
skip_64 = Dense(64, ...)(x) skip_64 = BatchNormalization()(skip_64)
```

Create the skip connection path for the first residual block. This is separate from the main processing path — the skip captures the linearly projected input at 64 dimensions, while the main path applies non-linear transformations. The Add() later sums these two paths.

```
x = Dense(32, ..., name='bottleneck')(x)
```

After the 64-unit residual block, compress to 32 units. This 'bottleneck' layer serves two purposes: (1) it forces the model to distill the most important information into a compact representation, and (2) it is the penultimate layer whose output is used as the tabular feature vector in fusion (via layers[-2] indexing).

```
loss='sparse_categorical_crossentropy'
```

Unlike the image and audio models which use categorical_crossentropy with one-hot labels, the tabular model uses sparse_categorical_crossentropy with integer labels. This is an implementation detail — both are mathematically equivalent, but sparse version is more memory efficient when not using one-hot encoding.

5.12 GlobalMultimodalGenerator (class)

Purpose	Batch generator for the fusion model that loads image, audio, and tabular data simultaneously for each triplet sample.

```
X_dict = {     'image_input':   np.array(X_img, ...),     'audio_input':
np.array(X_aud, ...),     'tabular_input': self.scaler.transform(...) }
```

Return inputs as a dictionary with keys matching the named Input layers of the global model ('image_input', 'audio_input', 'tabular_input'). Keras uses these keys to route each data array to the correct branch model input. The scaler.transform() is applied here — the scaler was fitted on training data during tabular preprocessing and is reused for consistency.

```
try: X_img.append(preprocess_chest_xray(row['img_path'])) except:
X_img.append(np.zeros((*IMG_SIZE, 3), dtype=np.float32))
```

Same silent error handling as in NIHImageGenerator. If an image file is corrupted, a black image is substituted. This allows training to continue even with a small number of bad files.

5.13 build_global_model()

Purpose	Connect the three pre-trained branch models, extract feature vectors, concatenate, and build the joint fusion head.

```
image_model.trainable = False audio_model.trainable = False
tabular_model.trainable = False
```

Freeze all three branch models entirely. During fusion training, only the new Dense(256), Dense(128), and Dense(4) layers will have their weights updated. This prevents the branch models from forgetting their individually-learned representations during joint training (catastrophic forgetting).

```
img_feat = img_branch.layers[-2].output aud_feat = aud_branch.layers[-2].output
tab_feat = tab_branch.layers[-2].output
```

Access the output tensor of the second-to-last layer of each branch. In all three models, layers[-1] is the final Dense(4, softmax) output and layers[-2] is the Dropout layer before it. This gives the raw feature vector before classification — richer than softmax probabilities for fusion. This is the core of late feature fusion.

```
merged = Concatenate(name='fusion_concat')([img_feat, aud_feat, tab_feat])
```

Concatenate all three feature vectors along the feature dimension. If image features are 1,024-dimensional, audio features are 1,280-dimensional (EfficientNetB0 output + GAP), and tabular features are 32-dimensional, the merged vector is 1,024 + 1,280 + 32 = 2,336-dimensional.

```
model = Model(     inputs=[img_branch.input, aud_branch.input,
tab_branch.input],     outputs=outputs, )
```

Create a unified Keras Model that takes three named inputs and produces one output. Keras automatically traces the computational graph from the three branch inputs through their respective frozen backbones, through the concatenation, and through the new fusion head to the output.

5.14 build_joint_doctor_report()

Purpose	Convert raw softmax probability output into a structured clinical report dictionary.

```
idx = int(np.argmax(probs)) final_class = MASTER_LE.inverse_transform([idx])[0]
```

Take the index of the highest probability class as the predicted class. inverse_transform converts the integer index back to the string class name ('asthma', 'copd', etc.). This is the inverse operation of MASTER_LE.transform().

```
'high_uncertainty': conf_pct < 65.0
```

Flag the prediction as high uncertainty if the maximum probability is below 65%. This threshold is a design choice — it corresponds to cases where the model is not clearly confident in any one class. In a real clinical deployment, this threshold should be calibrated on validation data to achieve a desired sensitivity/specificity trade-off.

```
'probabilities': {cls: round(float(p)*100, 2) for cls, p in zip(MASTER_CLASSES,
probs)}
```

Create a dictionary mapping each class name to its probability percentage. float() ensures numpy float32 values are serializable to JSON (numpy types are not JSON-serializable). round(x, 2) formats to 2 decimal places for readability.

5.15 plot_advanced_metrics()

Purpose	Generate ROC-AUC and Precision-Recall curves for all four classes on the test set.

```
y_true_oh = to_categorical(y_true, num_classes=NUM_CLASSES)
```

Convert integer class labels to one-hot encoding for multi-class ROC/PR computation. ROC and PR curves are inherently binary — to compute them for a multi-class problem, a one-vs-rest (OvR) approach is used: for class i, y_true_oh[:, i] is the binary label (1 if class i, 0 otherwise) and y_prob[:, i] is the predicted probability for class i.

```
fpr, tpr, _ = roc_curve(y_true_oh[:, i], y_prob[:, i]) roc_auc = auc(fpr, tpr)
```

Compute the ROC curve for class i: sweep the classification threshold from 1 to 0, computing False Positive Rate and True Positive Rate at each threshold. auc() computes the area under this curve using the trapezoidal rule. AUC=1.0 means perfect discrimination; AUC=0.5 means random performance.

```
precision, recall, _ = precision_recall_curve(y_true_oh[:, i], y_prob[:, i])
avg_p = average_precision_score(y_true_oh[:, i], y_prob[:, i])
```

Compute the Precision-Recall curve and its summary statistic, Average Precision (AP). AP is the area under the PR curve, weighted by recall changes — it rewards models that maintain high precision at high recall. Unlike AUC-ROC, AP is sensitive to class imbalance, making it a more informative metric for imbalanced medical datasets.

6. Technical Documentation

6.1 Environment and Dependencies

Library	Version Req.	Role
`tensorflow`	≥ 2.10	Deep learning framework, model training & inference
`keras`	via tf	High-level API for model building
`numpy`	≥ 1.23	Array operations, random number generation
`pandas`	≥ 1.5	Tabular data loading and manipulation
`scikit-learn`	≥ 1.2	Preprocessing, metrics, train/test split
`opencv-cv2`	≥ 4.6	Image loading, CLAHE, resize operations
`librosa`	≥ 0.9	Audio loading and mel spectrogram extraction
`matplotlib`	≥ 3.6	Training history and metrics visualization
`seaborn`	≥ 0.12	Confusion matrix heatmap rendering
`joblib`	≥ 1.2	Model and scaler serialization
`ydata-profiling`	≥4.0	Automated EDA report generation
`kaggle`	latest	Dataset download CLI

6.2 Training Configuration Summary

Optimizer	AdamW with weight_decay — decoupled L2 regularization
Image LR	1e-4 (lower because backbone layers are partially trainable)
Audio LR	5e-4 (higher because only classification head is trainable)
Tabular LR	5e-4
Global Fusion LR	1e-3 (highest — only the new fusion head layers are trained)
Loss (image/audio)	categorical_crossentropy (one-hot labels)
Loss (tabular)	sparse_categorical_crossentropy (integer labels)
Loss (fusion)	categorical_crossentropy
Batch Size	16 for all models
Max Epochs	50 with EarlyStopping
ES Patience (img)	4 epochs
ES Patience (tab)	12 epochs (tabular convergence is slower)

ES Patience (fus)	5 epochs
Class Weighting	compute_class_weight('balanced') applied to image and tabular models
Image Augmentation	rotation, shift, zoom, horizontal flip (training only)
Tabular Augmentation	MixUp with Beta(0.3, 0.3), 1200 synthetic samples

6.3 Model I/O Specification

Image Model

Input name	image_input
Input shape	(batch, 224, 224, 3) — float32, normalized [0, 1]
Output name	predictions (softmax)
Output shape	(batch, 4) — class probabilities summing to 1.0
Feature layer	layers[-2] output — (batch, 1024) after GlobalAveragePooling + Dropout

Audio Model

Input name	audio_input
Input shape	(batch, 224, 224, 3) — mel spectrogram as float32, [0, 1]
Output name	predictions (softmax)
Output shape	(batch, 4)
Feature layer	layers[-2] output — (batch, 1280) after GlobalAveragePooling + Dropout

Tabular Model

Input name	tabular_input
Input shape	(batch, 13) — StandardScaler normalized float32
Output name	class_probs (softmax)
Output shape	(batch, 4)
Feature layer	layers[-2] output — (batch, 32) bottleneck after Dense(32) + BatchNorm + GELU + Dropout

Global Fusion Model

Inputs	3 named inputs: image_input, audio_input, tabular_input
Input shapes	Same as individual branch models above
Output name	final_prediction
Output shape	(batch, 4)
Fusion vector	Concatenation of all three feature layers → (batch, ~2336) depending on backbone output sizes

6.4 Inference Guide

To run inference on a new patient using the saved global model:

23. Load the saved model: global_model = tf.keras.models.load_model('best_global_model.keras')
24. Load the tabular scaler: tabular_scaler = joblib.load('tabular_scaler.joblib')
25. Prepare image input: img = preprocess_chest_xray('path/to/xray.png') → np.expand_dims(img, 0)
26. Prepare audio input: aud = wav_to_melspectrogram('path/to/audio.wav') → np.expand_dims(aud, 0)
27. Prepare tabular input: tab = tabular_scaler.transform([[age, gender_enc, ...]]) — must be in TABULAR_FEATURES order
28. Run prediction: probs = global_model.predict({'image_input': img, 'audio_input': aud, 'tabular_input': tab})[0]
29. Generate report: report = build_joint_doctor_report(probs, patient_meta={'patient_id': 'P001', 'age': 45})

> ⚠ *Always use the same tabular_scaler that was fitted during training. Using a different scaler or no scaler will produce incorrect predictions because the model learned from scaled features.*

6.5 Known Limitations and Considerations

Data Alignment Limitation

The multimodal triplet construction pairs samples by class label only — not by actual patient. This means the fusion model learns inter-modal patterns based on class-level correlations, not true patient-level co-occurrence. For real clinical deployment, a dataset where all three modalities are collected from the same patients is required.

Audio Dataset Size

The ICBHI dataset is relatively small (hundreds of recordings). After filtering for the four target classes, the bottleneck triplet count per class may be very low (e.g., 10–50 samples for some classes). This limits fusion model training significantly.

NIH Label Mapping

The mapping of Atelectasis → asthma is clinically imprecise. Atelectasis (partial lung collapse) and asthma are distinct pathologies that can co-occur but are not equivalent. This mapping was made to enable a 4-class shared classification task.

Joblib for Keras Models

Using joblib.dump() on Keras Model objects is not officially supported and may produce inconsistent results across TensorFlow versions. The .keras and .h5 formats are the reliable persistence options for neural network models. The joblib save is provided for convenience but the .keras format should be preferred.

Class Imbalance in ICBHI

ICBHI is heavily imbalanced toward COPD and Healthy classes. Class weighting is not applied to the audio model training. Adding compute_class_weight() to audio training could improve recall for minority classes (asthma, pneumonia).

6.6 Output Files Reference

best_image_model.keras	Best DenseNet-121 checkpoint (by val_loss). Load with tf.keras.models.load_model().
final_image_model.keras	Model weights at training end (may differ from best checkpoint).
best_audio_model.keras	Best EfficientNet-B0 checkpoint.
final_audio_model.keras	Audio model at training end.
best_tabular_dnn.keras	Best residual MLP checkpoint.
final_tabular_dnn.keras	Tabular model at training end.
best_global_model.keras	Best fusion model checkpoint. Primary deployment artifact.
tabular_scaler.joblib	Fitted StandardScaler. Required for all tabular inference.
***.h5**	HDF5 versions of above models for legacy compatibility.
***.joblib**	Joblib-serialized model files (experimental for Keras models).
cm_*.png	Confusion matrix plots for each trained model.
history_*.png	Training/validation loss and accuracy curves.
advanced_metrics_*.png	ROC-AUC and Precision-Recall curve plots.
tabular_data_profile_report.html	ydata-profiling EDA report on the clinical dataset.